Change request log

Change 2

1. Concept Location

Step #	Description	Rationale
1	We ran the system	
2	We interacted with the system: after logging in we entered the schedule screen.	To get familiar with some of the features of the system, and identify the screens or graphical elements, we had to change. We found the Recent files under the file tab and interacted with it to know how the text is getting highlighted
3	We searched for "Recentfile" using the search tool of Intellij and we found private JSpinner recentFiles in GeneralOptionPane.java	Because we identified an option called “Recent file” under the file tab, within which a search bar to search the names of recent files was provided.
4	From the results, we wanted to search classes with RecentFiles, then we found the file “RecentFilesProvider.java"	We selected this file because there was only one “.java” filed named “RecentFilesProvider.java” , so we clicked on it.
5	We inspected the “RecentFilesProvider“	When we inspected the class, we found that the functionality of the class included Sorting Recent files, iterating over recent files, and creating a text field to filter recent files based on user input
6	We marked the “RecentFilesProvider.java” class as located	When we looked through the methods of the class, we found 2 methods which had the following functionalities: 1. Method Name: updateEveryTime Functionality: Returns a Boolean value indicating whether the menu should be updated every time it is displayed. 2. Method Name: update Functionality: Updates the given JMenu with a list of recent files, including dynamic filtering based on user input, sorting options, and the ability to open or clear recent files. Handles the creation of menu items, icons, tooltips, and separators. The functionality of the update method had to be changed in order to execute change request 2.

Time spent (in minutes): 30 minutes.

Classes and Methods:

- GeneralOptionPane.java

    - private JSpinner recentFiles;

- org/gjt/sp/jedit/menu/RecentFilesProvider.java
    - update
    - updateEveryTime

2. Impact Analysis

Step #	Description	Rationale
1	We inspected the method updateEveryTime, in the RecentfilesProvider class. And the method was discarded.	We realized this method need not have to be changed because this method only deals with the performance optimization by avoiding unnecessary updates whenever the menu is shown
2	We then inspected the "update" method and marked it to be changes	Because the class deals with the following functionalities: • Retrieving the current view and setting up listeners for actions and changes. • Retrieving a list of recent files from BufferHistory.getHistory(). • Handling the case when there are no recent files, adding a disabled menu item with a corresponding message. • Creating a text field for dynamically filtering recent files based on user input. • Sorting recent files alphabetically if the sorting configuration is enabled. • Iterating over recent files, creating menu items with icons and tooltips, and adding them to the menu. • Adding an action listener to each menu item to open the selected file when clicked. • Providing a text field listener for dynamically filtering based on user input. • Adding a menu item to clear the recent files history. • Handling error logging for pattern syntax exceptions.

Time spent (in minutes): 45 minutes.

Classes and Methods:

- org/gjt/sp/jedit/menu/RecentFilesProvider.java
    - update
    - updateEveryTime

3. Actualization

Step #	Description	Rationale
1	While we inspected the class, we found that we had to make change to the addKeyListener	The function of add keylistener when a key is released, it retrieves the text from the field, dynamically filters menu items based on the entered text, and enables or disables them accordingly. If the entered text forms a valid regex pattern, it uses it for matching; otherwise, it catches and logs a PatternSyntaxException. The filtering considers whether the text contains wildcard characters (*'' or '?') and appends "* if not present, supporting an old style of matching the start of a file name.

2	We found an if condition, which was used to filter the search results	if ((! typedText.contains("*")) && (! typedText.contains("?")))
		{
		// Old style (before jEdit 4.3pre18): Match start of file name
		regex = regex + "*";
		}
		
		We understood that this code snippet presents in the if condition was used to match the keyword to the start of the file. We decided to change this snippet.

3	Altering the if condition using regex, to match any position of the keyword in the file name.	We replaced the existing regex with String regex = "." + Pattern.quote(typedText) + ".";**:** This Constructs a regex pattern (regex) based on the user-entered text (typedText). It surrounds the typed text with dots and uses Pattern.quote() to escape any special characters in the typed text. This construction suggests that the pattern is aiming to match the typed text as a whole word within a larger context. Also in the pattern.compile statement we remove the 'StandardUtilities.globToRE()' as we converted the regex in desired format that doesn't require any additional processing.
4	We encountered an error	When we tried running the JEdit software after making this change to the regex, we found that, this expression altered the functionality such that even when the start of the title had the searched keyword, the title was not highlighted.
5	We again tried altering the regular expression to String regex = ".*" + Pattern.quote(typedText) + ".*";**:**	After adding ".*" , to the regex we were able to successfully implement the change.
6	Performed multiple tests to ensure that the search feature is incorporated properly	We uploaded new text files which have 'the' substring at first, middle and at the end of file name. Then, we opened it to appear in recent files and evaluated the change

Time spent (in minutes): 90 minutes.

Classes and Methods: (Inspected)

- org/gjt/sp/jedit/menu/RecentFilesProvider.java
    - update

Classes and Methods: (Changed)

- org/gjt/sp/jedit/menu/RecentFilesProvider.java
    - update

4. Validation

Step #	Testing Level	Description	Rationale	Result
1	Unit Testing	Unit test the update method for constructing the regex pattern.	Ensure accurate construction of the regex pattern to match the string anywhere in the file name.	Pass- the regex pattern is constructed accurately.
2	Unit Testing	Unit test the regex pattern matching functionality.	Verify correct highlighting of file names containing the given string anywhere in their name.	Pass - highlighting occurs as expected.
3	Regression Testing	Regression test recent files functionality after the change.	Ensure existing features work correctly with the modified highlighting.	Pass - recent files functionality remains intact.
4	Regression Testing	Regression test performance after the change.	Confirm that the recent files menu remains responsive and performs well.	Pass - performance is not negatively impacted.
5	System Testing	System test recent files menu with modified highlighting.	Evaluate overall functionality of the recent files menu, focusing on the new highlighting behavior.	Pass - recent files menu works seamlessly with correct highlighting.
6	System Testing	System test recent files menu with various scenarios (strings, numbers and special characters).	Confirm modified highlighting handles different file names and edge cases correctly.	Pass - highlighting behaves appropriately in various scenarios.
7	Integration Testing	Integration test recent files menu with new highlighting.	Confirm recent files menu integrates well with updated highlighting and interaction is smooth.	Pass - recent files menu and highlighting work together seamlessly.
8	Integration Testing	Integration test recent files menu with other jEdit features.	Ensure new highlighting does not conflict with other features.	Pass - no conflicts or issues with other features are observed.
9	Acceptance Testing	Acceptance test recent files menu with updated highlighting.	Evaluate that the new highlighting feature is performed as per the change asked.	Pass - the new highlighting feature performed as per the change asked
10	Acceptance Testing	Acceptance test entire jEdit application with updated recent files functionality.	Confirm overall jEdit experience is improved with modified highlighting.	Pass.

Time spent (in minutes): 100 minutes.

- org/gjt/sp/jedit/menu/RecentFilesProvider.java
    - update

5. Summary of the change request

Phase	Time (minutes)	No. of classes inspected	No. of classes changed	No. of methods inspected	No. of methods changes
Concept location	30	2	1	2	1
Impact Analysis	45	1	1	2	1
Actualization	90	1	1	1	1
Validation	100	2	1	2	1
Total	265	2	1	2	1

6. Conclusions

The change request aimed to enhance the functionality of the text box in the File » Recent Files main menu of jEdit. The current behavior highlighted recent files only when the entered string matched the beginning of a file name. The requested modification was to broaden this behavior, enabling the highlighting of files containing the entered string anywhere in their names. While conceptually straightforward, the implementation posed challenges due to the existing code structure. The need to adjust the regex pattern used for matching required careful consideration to avoid unintended side effects. Additionally, ensuring that the modification did not negatively impact other aspects of the recent files feature required thorough testing. Coordination with the existing codebase and understanding the nuances of the regex matching logic were crucial aspects of addressing this change request. Despite these challenges, successful implementation of the modification enhanced the user experience by providing more flexible and intuitive filtering of recent files.